Department of Computer Science and Engineering
NITK Surathkal

Database Management Systems Project

Academic Year 2025 - 26

NEXUS EXCHANGE ENGINE

High-Integrity Digital Asset Trading Platform

Submitted To:

Saumya Hegde Ma'am

Submitted By:

241CS104 Ajitesh Kumar Kallepali
241CS114 Darshan Sai Naath
241CS159 Tanay Nahta
241CS165 Vishal Murugan

Table of Contents

Table of Contents ..... 2 - 3

1. Introduction 4

2. Literature Survey 5

    2.1 ACID Transactions and Concurrency Control .....

    2.2 Order-Matching Engines .....

    2.3 Database Normalization .....

    2.4 Immutable Audit Logs in Financial Systems .....

    2.5 Weighted Average Cost (WAC) Accounting .....

3. Objectives 6

4. Features ..... 7 - 8

    4.1 Core Trading Features .....

    4.2 Portfolio & Analytics Features .....

    4.3 Security & Audit Features .....

    4.4 Administrative Features .....

5. Requirement Specification ..... 9 - 10

    5.1 Functional Requirements .....

    5.2 Non-Functional Requirements .....

    5.3 Hardware & Software Requirements .....

6. System Design 11

    6.1 Architecture Overview .....

    6.2 Service Layer Components .....

7. Database Schema Design ..... 12 - 17

    7.1 Entity Overview .....

    7.2 Table Definitions .....

        7.2.1 users .....

        7.2.2 assets .....

        7.2.3 wallets .....

        7.2.4 holdings .....

        7.2.5 orders .....

        7.2.6 trades .....

        7.2.7 audit_log .....

        7.2.8 price_history .....

        7.2.9 ledger_entries .....

    7.3 Database Views .....

    7.4 Stored Procedures .....

    7.5 Triggers .....

8. Entity-Relationship (ER) Diagram ..... 18 - 19

8.1 Entities and Relationships .....

9. Implementation ..... 20 - 21

9.1 Tools and Technologies .....

9.1.1 Frontend .....

9.1.2 Backend .....

9.1.3 Database .....

9.2 Key Implementation Decisions .....

10. References 22

1. Introduction

The financial technology landscape demands trading systems capable of handling simultaneous high-frequency transactions while maintaining absolute data integrity. Traditional brokerage platforms frequently encounter issues such as race conditions, double-spending anomalies, and inconsistent portfolio states under concurrent load i.e., shortcomings that erode user trust and regulatory compliance.

The Nexus Exchange Engine addresses these challenges head-on. It is a high-integrity digital asset trading platform engineered to ensure accurate and secure asset exchanges even during simultaneous transactions. The system features an automated order-matching engine, real-time portfolio analytics, and a reliability-first architecture built on proven database principles.

At its core, Nexus Exchange Engine is a full-stack web application comprising a React-based dashboard frontend, a Node.js/Express API gateway, and a MySQL 8 InnoDB relational database. The database design leverages ACID-compliant stored procedures, row-level locking, immutable audit triggers, views, and Third Normal Form (3NF) schema design to guarantee correctness under all operating conditions.

This report documents the complete system design, requirement analysis, database schema, ER diagram, and technology choices made during the development of the system.

2. Literature Survey

The design of Nexus Exchange Engine draws from well-established research and industry practice across several domains:

2.1 ACID Transactions and Concurrency Control

Gray and Reuter (1992) in Transaction Processing: Concepts and Techniques establish the theoretical foundation for ACID guarantees i.e., Atomicity, Consistency, Isolation, and Durability, which remain the gold standard for financial database systems. Their treatment of two-phase locking (2PL) and deadlock avoidance directly informs the canonical lock ordering strategy implemented in the execute_trade stored procedure of this system.

2.2 Order-Matching Engines

Leber, Bilokon, and Burgess (2011) in Building Low-Latency Applications with C++ describe the architecture of exchange-grade order-matching engines, emphasising price-time priority queues. While Nexus Exchange Engine targets educational scale rather than nanosecond latency, the same logical matching semantics, i.e., MARKET orders consuming the best available LIMIT resting orders that are faithfully reproduced at the stored-procedure level.

2.3 Database Normalization

Codd's seminal work on relational database theory (1970, 1972) introduced the Normal Forms. The schema design in Nexus Exchange Engine achieves Third Normal Form (3NF), eliminating all transitive dependencies and ensuring minimal data redundancy while preserving referential integrity through foreign key constraints.

2.4 Immutable Audit Logs in Financial Systems

The Payment Card Industry Data Security Standard (PCI-DSS) and various financial regulatory frameworks mandate tamper-evident audit trails. Research by Merkle (1987) on cryptographic hash chains inspired the approach used in Nexus Exchange Engine, where every trade event is hashed using SHA-512 and stored immutably via an AFTER INSERT trigger, preventing post-hoc modification.

2.5 Weighted Average Cost (WAC) Accounting

The weighted average cost method for inventory/asset valuation is prescribed under IAS 2 (International Accounting Standards) and widely adopted in portfolio systems. The wac_view in the system computes real-time WAC across buy events, providing accurate cost-basis reporting for each user's holdings.

3. Objectives

The primary goal of the Nexus Exchange Engine is to show that a relational database, if implemented correctly, can be the core of the integrity system of a financial trading system. Objectives include:

- Designing and implementing a database management system-centred trading platform that can enforce ACID via the database level, not just the application level.
- Developing an automated order-matching engine that uses stored procedures to execute deadlock-safe locking strategies, including support for partial order fills.
- Delivering real time portfolio analytics with Mark-to-Market (MTM) valuation and Weighted Average Cost (WAC) reporting via views in the database.
- Delivering an immutable crypto-hashed audit trail via triggers in the database system that is tamper-evident and protected against modification at the database level.
- Demonstrate how row-level locking (SELECT FOR UPDATE) and REPEATABLE READ isolation can prevent ghost reads and dirty writes in concurrent write scenarios.
- Deliver a 3NF-compliant relational schema that eliminates redundancy while maximizing query performance.
- Provide a complete order lifecycle management system including order placement, matching, partial fills, and cancellation — all enforced at the stored procedure level.
- Deliver a clean, easy to use React dashboard that mirrors the live state of the database, showing portfolio holdings, price charts, order form and admin views.

4. Features

4.1 Core Trading Features

Feature	Description
MARKET Orders	Instantly executed against the best available price in the order book.
LIMIT Orders	Placed at a specific price; matched when conditions are met.
BUY / SELL Operations	Full support for both sides of a trade with pre-trade validation.
Automated Order Matching	Stored procedure matches incoming orders against resting orders atomically.
Partial Order Fills	settle_partial_trade procedure handles orders that match only part of their requested quantity, updating remaining quantity and keeping order OPEN until fully filled.
Order Cancellation	cancel_order stored procedure cancels OPEN orders and atomically refunds reserved cash (BUY) or asset quantity (SELL) back to the user.
Concurrent Trade Safety	Canonical lock ordering prevents deadlocks; CHECK constraints prevent negative balances.
Live Order Book	get_order_book procedure returns all open LIMIT orders for a given asset sorted by price-time priority (BUY side DESC, SELL side ASC).

4.2 Portfolio & Analytics Features

Feature	Description
Mark-to-Market (MTM) Valuation	portfolio_mtm view joins holdings with live asset prices for real-time P&L.
Weighted Average Cost (WAC)	wac_view calculates average cost basis across all buy events per user/asset.
Exchange Capital Distribution	user_exchange_weight_view computes each user's total portfolio value and their percentage share of the total capital held across the entire exchange, using an unpartitioned SUM() OVER () window function.
P&L Badges	Holdings table shows unrealised gain/loss colour-coded per position.
User P&L Summary	user_pnl_summary_view aggregates total portfolio value, unrealised P&L, and percentage gain/loss per user using window functions (RANK()).
Asset Trading Volume	asset_volume_view shows total quantity traded and total value traded per asset, computed via GROUP BY aggregations on the trades table.
User Trade Statistics	user_trade_summary_view provides per-user statistics: total trades, total volume, and most traded asset.
Running Balance History	running_balance_view uses SUM() OVER window function on ledger_entries to show how each user's wallet balance evolved over every transaction.
Price Chart	Recharts-powered line chart showing historical asset price movements sourced from price_history.
Portfolio Summary Bar	Aggregate metrics: total value, day change, number of positions.

4.3 Security & Audit Features

Feature	Description
JWT Authentication	Stateless JSON Web Token auth with expiry; all API routes are protected.
Zod Input Validation	Schema-level validation on all incoming API payloads.
Immutable Audit Log	AFTER INSERT trigger writes SHA-512 hash of every trade; log is write-once.
Tamper-Prevention Trigger	prevent_audit_tampering trigger fires BEFORE UPDATE or DELETE on audit_log and immediately raises an error, enforcing write-once semantics at the database level.
WORM Audit Feed	Admin view shows live cryptographic audit trail with transaction hashes.
Row-Level Locking	SELECT FOR UPDATE on wallets and holdings during every trade execution.

4.4 Administrative Features

Feature	Description
Admin Dashboard	Separate admin tab showing system health, lock states, and audit feed.
Lock Monitor	Displays active database locks and potential contention points.
User Management	Admin can view all registered users and their account states.
Paginated Trade History	Full ledger of all trades with filtering by user, asset, and date range.
Deposit & Withdrawal	deposit_funds and withdraw_funds stored procedures allow admin-managed wallet funding and withdrawal, each generating corresponding ledger entries.
Open Orders View	open_orders_view surfaces all currently pending LIMIT orders across all users with asset symbol, price, and quantity.

5. Requirement Specification

5.1 Functional Requirements

FR-01: User Registration & Authentication

Users shall be able to register with a unique username and email. Passwords shall be stored as salted hashes. Authentication shall produce a JWT valid for a configurable duration.

FR-02: Asset Catalogue

The system shall maintain a catalogue of tradable digital assets, each with a current price updated by administrators or market feeds. Every price update shall be automatically recorded in price_history via a database trigger.

FR-03: Wallet Management

Each user shall have a fiat wallet (cash balance). The system shall reject any transaction that would reduce balance below zero via database CHECK constraints. Administrators shall be able to deposit or withdraw funds via dedicated stored procedures.

FR-04: Order Placement

Users shall be able to place MARKET and LIMIT orders for BUY and SELL operations. Order validation must occur before execution.

FR-05: Order Matching

The execute_trade stored procedure shall match incoming orders against resting opposite-side orders atomically, completing the trade or placing the order in a pending state. The settle_partial_trade procedure shall handle cases where available quantity is less than the requested quantity.

FR-06: Order Cancellation

Users shall be able to cancel any OPEN order. The cancel_order stored procedure shall atomically revert reserved funds or asset quantities and update the order status to CANCELLED.

FR-07: Order Book

The get_order_book stored procedure shall return the current state of open LIMIT orders for any asset, sorted by price-time priority.

FR-08: Portfolio View

Users shall see real-time portfolio holdings with MTM valuation, WAC cost basis, and unrealised P&L.

FR-09: Trade History

Users shall be able to view a paginated history of all their executed trades.

FR-10: Audit Log

Every executed trade shall be recorded in an immutable audit log with a SHA-512 cryptographic hash by a database trigger. The audit log shall be protected against modification by a BEFORE UPDATE/DELETE trigger that raises an error on any tampering attempt.

FR-11: Exchange Capital Distribution

The system shall provide a view (user_exchange_weight_view) that computes each user's total portfolio value and their proportional share of total capital held across the entire exchange. This shall be computed using an unpartitioned SUM() OVER () window function, enabling market dominance analysis without requiring application-layer aggregation.

5.2 Non-Functional Requirements

Category	Requirement
Integrity	All trades must be ACID-compliant; no partial updates permitted.
Concurrency	Concurrent trades on the same asset must not produce race conditions or double-spending.
Security	All API endpoints must require valid JWT. Passwords must be hashed (bcrypt).
Normalization	Database schema must satisfy Third Normal Form (3NF).
Auditability	Audit log must be tamper-evident; no UPDATE/DELETE on audit records — enforced by trigger.
Usability	UI must be responsive and usable on desktop browsers.
Maintainability	Codebase must be modular: separate service, route, and data-access layers.

5.3 Hardware & Software Requirements

Component	Specification
Operating System	macOS / Linux / Windows (with WSL)
Runtime	Node.js v20+ (LTS)
Database	MySQL 8.0+ (InnoDB engine)
Frontend Build Tool	Vite 5+
Browser	Chrome / Firefox / Edge (modern versions)
Memory	Minimum 4 GB RAM for development
Storage	Minimum 1 GB available disk space

6. System Design

6.1 Architecture Overview

The system follows a layered three-tier architecture:

Tier	Technology	Responsibility
Presentation	React 18 + Vite + Tailwind CSS	User interface, charts, forms, real-time display
Application	Node.js + Express.js (port 3001)	API gateway, business logic, auth, validation
Data	MySQL 8 InnoDB (port 3306)	Persistent storage, ACID transactions, triggers, views, stored procedures

The frontend communicates with the backend exclusively via HTTP/JSON REST API. The backend connects to MySQL through a connection pool (mysql2). All trade execution logic resides in stored procedures at the database layer to ensure atomicity regardless of application-layer failures.

6.2 Service Layer Components

Service	Responsibilities
Trade Service	Invokes execute_trade and settle_partial_trade stored procedures; handles pre-trade validation and post-trade response.
Order Service	Handles order placement, cancel_order procedure invocation, and get_order_book queries.
Portfolio Service	Queries portfolio_mtm, wac_view, user_pnl_summary_view, asset_volume_view, and user_exchange_weight_view; aggregates MTM, WAC, P&L, and exchange capital distribution data.
User Service	User registration, password hashing, JWT issuance, deposit_funds and withdraw_funds procedure invocation.
Audit Service	Reads from the immutable audit_log table; supports filtering and pagination.

7. Database Schema Design

7.1 Entity Overview

The Nexus Exchange Engine database (nexus_db) is designed in Third Normal Form (3NF). It consists of the following primary entities:

Table	Description
users	Stores registered user accounts, credentials, and roles.
assets	Catalogue of tradable digital assets with current prices.
wallets	Fiat cash balances for each user (one-to-one with users).
holdings	Quantity of each asset held by each user.
orders	All orders placed (MARKET/LIMIT, BUY/SELL, pending/filled/cancelled).
trades	Executed trade records linking buyer, seller, asset, price, and quantity.
audit_log	Immutable, cryptographically hashed record of every trade event.
price_history	Historical price snapshots for each asset, recorded over time.
ledger_entries	Double-entry financial ledger recording every debit and credit resulting from a trade (wallet and holdings movements).

7.2 Table Definitions

7.2.1 users

Key	Column Name	Type / Constraints	Description
PK	**id**	INT, AUTO_INCREMENT, NOT NULL	Unique user identifier
AK	**email**	VARCHAR(255), UNIQUE, NOT NULL	User email address (alternate key)
	password_hash	VARCHAR(255), NOT NULL	bcrypt-hashed password
	name	VARCHAR(255), NOT NULL	Display name of the user
	created_at	TIMESTAMP, DEFAULT NOW()	Account creation timestamp

7.2.2 assets

Key	Column Name	Type / Constraints	Description
PK	**id**	INT, AUTO_INCREMENT, NOT NULL	Unique asset identifier
AK	**symbol**	VARCHAR(10), UNIQUE, NOT NULL	Ticker symbol (e.g. BTC, ETH)
	name	VARCHAR(100), NOT NULL	Full asset name
	current_price	DECIMAL, NOT NULL	Live market price
	updated_at	TIMESTAMP, DEFAULT NOW()	Price last-updated timestamp

7.2.3 wallets

Key	Column Name	Type / Constraints	Description
PK	**id**	INT, AUTO_INCREMENT, NOT NULL	Unique wallet identifier
FK, AK	**user_id**	INT, FK → users.id, UNIQUE	Owner reference — unique enforces 1:1 with users
	balance	DECIMAL, NOT NULL	Available fiat cash balance
	updated_at	TIMESTAMP, DEFAULT NOW()	Balance last-updated timestamp

7.2.4 holdings

Key	Column Name	Type / Constraints	Description
PK	**id**	INT, AUTO_INCREMENT, NOT NULL	Unique holding identifier
FK, AK	**user_id**	INT, FK → users.id, part of composite UK	Owner reference
FK, AK	**asset_id**	INT, FK → assets.id, part of composite UK	Asset reference
	quantity	DECIMAL, NOT NULL	Units of the asset currently held
	avg_cost_basis	DECIMAL, NOT NULL	Weighted average purchase cost per unit
	updated_at	TIMESTAMP, DEFAULT NOW()	Last updated timestamp

7.2.5 orders

Key	Column Name	Type / Constraints	Description
PK	**id**	INT, AUTO_INCREMENT, NOT NULL	Unique order identifier
FK	**user_id**	INT, FK → users.id, NOT NULL	User who placed the order
FK	**asset_id**	INT, FK → assets.id, NOT NULL	Asset being traded
	type	ENUM('BUY','SELL'), NOT NULL	Trade direction
	order_type	ENUM('MARKET','LIMIT'), NOT NULL	Order classification
	qty	DECIMAL, NOT NULL	Requested quantity
	filled_qty	DECIMAL, DEFAULT 0	Quantity filled so far (for partial fills)
	limit_price	DECIMAL, NULLABLE	Target price — only set for LIMIT orders
	status	ENUM('OPEN','FILLED','CANCELLED'), DEFAULT 'OPEN'	Order lifecycle state
	created_at	TIMESTAMP, DEFAULT NOW()	Order placement timestamp

7.2.6 trades

Key	Column Name	Type / Constraints	Description
PK	**id**	INT, AUTO_INCREMENT, NOT NULL	Unique trade identifier
FK	**buy_order_id**	INT, FK → orders.id, NOT NULL	The matched buy-side order
FK	**sell_order_id**	INT, FK → orders.id, NOT NULL	The matched sell-side order
FK	**asset_id**	INT, FK → assets.id, NOT NULL	Asset that was exchanged
	qty	DECIMAL, NOT NULL	Quantity exchanged in this trade
	executed_price	DECIMAL, NOT NULL	Price at which the trade was executed
	executed_at	TIMESTAMP, DEFAULT NOW()	Trade execution timestamp

7.2.7 audit_log

Key	Column Name	Type / Constraints	Description
PK	**id**	INT, AUTO_INCREMENT, NOT NULL	Unique audit entry identifier
FK	**trade_id**	INT, FK → trades.id, NOT NULL	Trade that this audit entry covers
	tx_hash	VARCHAR(130), NOT NULL	SHA-512 cryptographic hash of the trade record
	created_at	TIMESTAMP, DEFAULT NOW()	Timestamp when the audit entry was logged

7.2.8 price_history

Key	Column Name	Type / Constraints	Description
PK	**id**	INT, AUTO_INCREMENT, NOT NULL	Unique price record identifier
FK	**asset_id**	INT, FK → assets.id, NOT NULL	Asset this price belongs to
	price	DECIMAL, NOT NULL	Recorded price at the snapshot time
	recorded_at	TIMESTAMP, DEFAULT NOW()	When this price was captured

7.2.9 ledger_entries

Key	Column Name	Type / Constraints	Description
PK	**id**	INT, AUTO_INCREMENT, NOT NULL	Unique ledger entry identifier
FK	**trade_id**	INT, FK → trades.id, NULLABLE	Trade that generated this entry (NULL for deposits/withdrawals)

Key	Column Name	Type / Constraints	Description
FK	**user_id**	INT, FK → users.id, NOT NULL	User whose balance is affected
	entry_type	ENUM('DEBIT','CREDIT'), NOT NULL	Direction of the financial movement
	amount	DECIMAL, NOT NULL	Monetary value of the movement
FK	**asset_id**	INT, FK → assets.id, NULLABLE	Asset involved (NULL for fiat-only movements)
	created_at	TIMESTAMP, DEFAULT NOW()	Entry creation timestamp

7.3 Database Views

View	Tables Used	Purpose
portfolio_mtm	holdings JOIN assets (via asset_id)	Computes Mark-to-Market portfolio value: quantity × current_price per holding per user.
wac_view	holdings (avg_cost_basis column)	Exposes weighted average cost basis per user-asset pair for P&L calculation.
open_orders_view	orders JOIN assets (via asset_id)	Returns all OPEN orders across all users with asset symbol, order type, side, quantity, and limit price.
user_pnl_summary_view	holdings JOIN assets	Aggregates per-user total portfolio value, total cost basis, unrealised P&L, and percentage gain/loss. Uses RANK() window function to rank users by portfolio value.
asset_volume_view	trades JOIN assets	Computes total quantity traded and total value traded (SUM(qty × executed_price)) per asset using GROUP BY.
user_trade_summary_view	trades JOIN ledger_entries JOIN users	Per-user trading statistics: total trades executed, total traded volume, and most traded asset identified via a correlated subquery.
running_balance_view	ledger_entries	Uses SUM(amount) OVER (PARTITION BY user_id ORDER BY created_at) to compute a running wallet balance for each user across every ledger event.
user_exchange_weight_view	holdings JOIN assets JOIN users	Computes each user's total portfolio value and their percentage share of the total capital held across the entire exchange. Uses an unpartitioned SUM() OVER () window function (no PARTITION clause) to compute the global exchange total, then divides each user's value by that total to derive their exchange weight percentage.

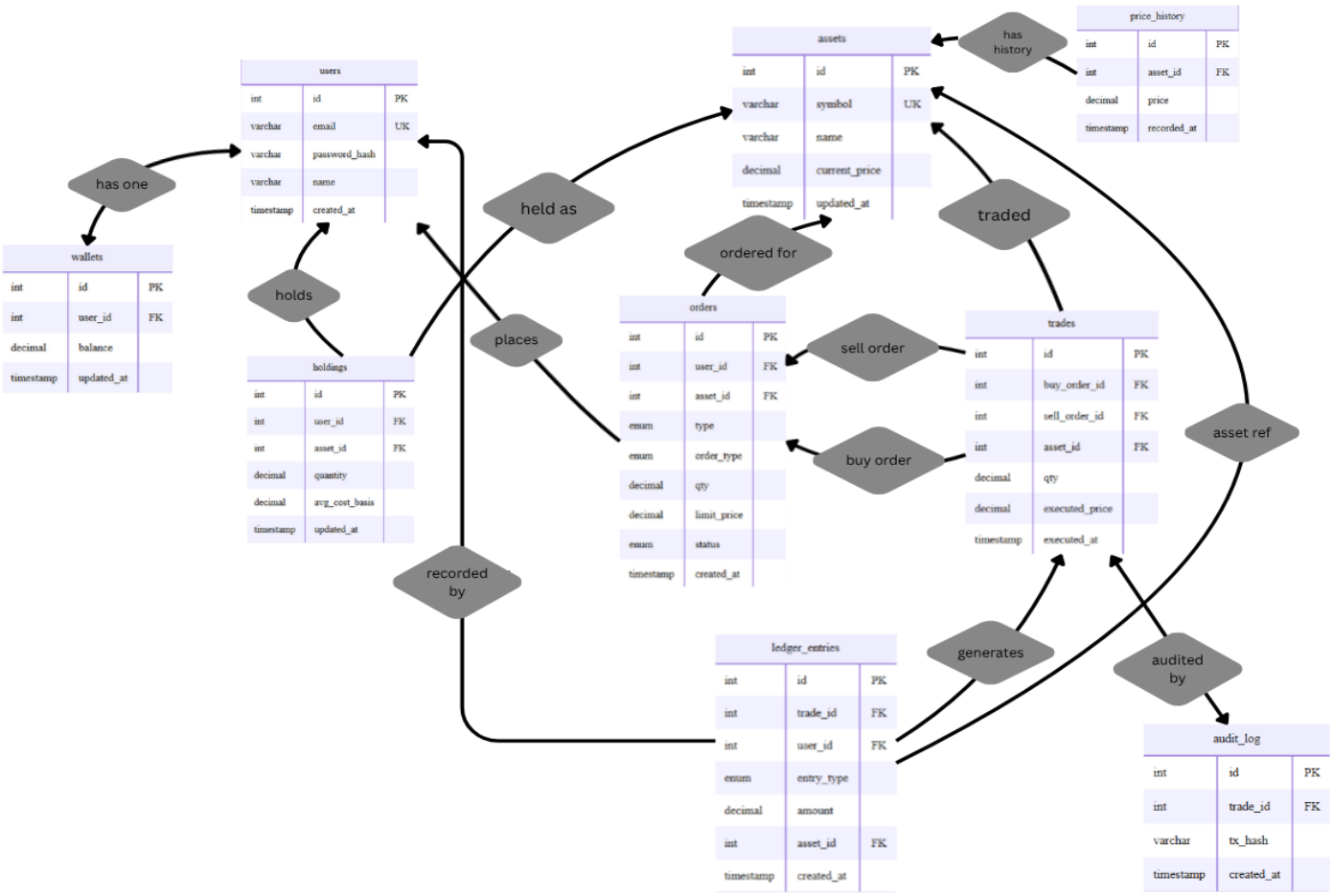
7.4 Stored Procedures

Procedure	Description
execute_trade(buy_order_id, sell_order_id, asset_id, qty, price)	Atomic trade execution wrapped in BEGIN...COMMIT. Acquires SELECT FOR UPDATE locks on both wallets in canonical user_id order (lower id first) to prevent deadlocks. Debits buyer wallet, credits seller wallet, updates holdings avg_cost_basis for both parties, inserts into trades and ledger_entries. Rolls back on any constraint violation.
settle_partial_trade(buy_order_id, sell_order_id, asset_id, available_qty, price)	Handles partial fills where available_qty < requested qty. Executes trade for available_qty, increments filled_qty on the buy order, and keeps the order status as OPEN until fully filled. Uses the same canonical locking strategy as execute_trade.
cancel_order(order_id)	Cancels an OPEN order atomically. For BUY orders, refunds the reserved cash (qty × limit_price) back to the user's wallet and inserts a CREDIT ledger entry. For SELL orders, restores the reserved asset quantity to holdings. Sets order status to CANCELLED. Rolls back if the order is not in OPEN state.
deposit_funds(user_id, amount)	Credits the specified amount to a user's wallet balance. Validates that amount > 0. Inserts a CREDIT entry into ledger_entries with a NULL trade_id to represent a standalone deposit event.
withdraw_funds(user_id, amount)	Debits the specified amount from a user's wallet. Validates sufficient balance via SELECT FOR UPDATE before modification. Inserts a DEBIT entry into ledger_entries with NULL trade_id. Rolls back if the resulting balance would be negative.
get_order_book(asset_id)	Returns the live order book for a given asset: all OPEN LIMIT orders, BUY side sorted by limit_price DESC (highest bids first), SELL side sorted by limit_price ASC (lowest asks first), implementing price-time priority semantics.

7.5 Triggers

Trigger	Event	Action
audit_log_trigger	AFTER INSERT on trades	Computes SHA2(CONCAT(trade fields), 512) and inserts the 130-character hash as tx_hash into audit_log. Ensures every trade has an immutable cryptographic fingerprint.
prevent_audit_tampering	BEFORE UPDATE or BEFORE DELETE on audit_log	Immediately signals SQLSTATE '45000' with message 'Audit log is immutable'. Blocks any modification or deletion of audit records at the database level, regardless of the caller.
price_history_trigger	AFTER UPDATE on assets (when current_price changes)	Automatically inserts a new row into price_history capturing the updated price and current timestamp. Ensures price history is always complete without relying on application-layer code.

Nexus Exchange E-R Diagram



9. Implementation

9.1 Tools and Technologies

9.1.1 Frontend

Technology	Version	Purpose
React	18+	Component-based UI framework for building the trading dashboard.
Vite	5+	Lightning-fast frontend build tool and development server (port 5173).
Tailwind CSS	3+	Utility-first CSS framework for rapid, consistent UI styling.
Recharts	Latest	Declarative charting library for the asset price line chart and portfolio analytics.
Axios	Latest	HTTP client with JWT interceptor for all API requests.

9.1.2 Backend

Technology	Version	Purpose
Node.js	18+ LTS	JavaScript runtime for the server-side application.
Express.js	4+	Minimal web framework for REST API routing and middleware.
mysql2	Latest	MySQL driver with connection pooling and Promise support.
jsonwebtoken	Latest	JWT creation and verification for authentication.
bcrypt	Latest	Password hashing with salting for secure credential storage.
Zod	Latest	Schema-based input validation on all API request bodies.

9.1.3 Database

Technology	Version	Purpose
MySQL	8.0+	Primary relational database management system.
InnoDB	Built-in	Storage engine with ACID transactions, row-level locking, and FK support.
Stored Procedures	SQL	execute_trade, settle_partial_trade, cancel_order, deposit_funds, withdraw_funds, get_order_book: full order lifecycle management at the DB layer.
Triggers	SQL	audit_log_trigger (SHA-512 audit), prevent_audit_tampering (write-once enforcement), price_history_trigger (automatic price recording).
Views	SQL	portfolio_mtm, wac_view, open_orders_view, user_pnl_summary_view, asset_volume_view, user_trade_summary_view, running_balance_view, user_exchange_weight_view.

9.2 Key Implementation Decisions

Stored Procedure for Trade Execution

All trade logic is placed inside stored procedures (execute_trade, settle_partial_trade) rather than in application code. This guarantees atomicity even if the application server crashes mid-transaction. The procedures use SELECT FOR UPDATE to lock wallet and holdings rows before modification.

Partial Fill Support via settle_partial_trade

Real exchange order books commonly encounter situations where a single order cannot be fully satisfied by one counter-order. The settle_partial_trade procedure handles this by executing the trade for the available quantity, updating filled_qty on the order, and keeping the order OPEN until it is fully satisfied. This mirrors the behaviour of production matching engines.

Atomic Order Cancellation

The cancel_order procedure ensures that cancelling an order is never a partial operation. It acquires the necessary locks, reverts reserved balances, inserts a compensating ledger entry, and updates order status — all within a single transaction.

Complete Ledger Coverage via deposit_funds and withdraw_funds

The ledger_entries table is populated not only by trades but also by standalone deposit and withdrawal operations. This means the running_balance_view provides a complete, unbroken financial history of every wallet movement, not just trade-driven ones.

Canonical Lock Ordering for Deadlock Prevention

When a trade involves two users, the procedure always locks the user with the lower user_id first. This canonical ordering ensures that two concurrent trades involving the same pair of users can never deadlock each other, as they will always contend for locks in the same sequence.

REPEATABLE READ Isolation Level

MySQL InnoDB defaults to REPEATABLE READ, which prevents dirty reads and non-repeatable reads. Combined with SELECT FOR UPDATE, this ensures that the balance or holdings quantity seen at the start of the transaction will not change underneath the procedure during execution.

SHA-512 Audit Hashing via Trigger

The audit_log_trigger fires AFTER INSERT on the trades table. It computes SHA2(CONCAT(trade fields), 512) and writes the result to audit_log. The prevent_audit_tampering trigger then guards this table by raising a database-level error on any UPDATE or DELETE attempt, making the audit trail truly write-once and tamper-evident regardless of application logic.

Automated Price History via Trigger

The price_history_trigger fires AFTER UPDATE on the assets table whenever current_price changes. This ensures price_history is always populated automatically and consistently, without depending on the application layer to remember to log it.

Exchange Capital Distribution via Window Function

The user_exchange_weight_view uses an unpartitioned SUM() OVER () window function — deliberately omitting any PARTITION BY clause — to compute the aggregate portfolio value across all users

simultaneously. Each user's individual portfolio value is then divided by this global total to yield their exchange weight percentage. This demonstrates that a single SQL view can answer a market-level analytical question (analogous to Bitcoin dominance on a real exchange) without any application-layer aggregation logic.

Advanced Analytics via Window Functions

The running_balance_view and user_pnl_summary_view use SQL window functions (SUM() OVER and RANK()) to compute running totals and user rankings directly in the database. The user_exchange_weight_view further extends this with a global SUM() OVER () (no partition), demonstrating the full range of partitioned and unpartitioned window function usage.

Connection Pooling

The backend uses a mysql2 connection pool (db/pool.js) rather than single connections. Pooling reduces connection overhead and prevents the database from being overwhelmed by concurrent API requests, each of which may spawn a short-lived transaction.

10. References

[1] Gray, J., & Reuter, A. (1992). Transaction Processing: Concepts and Techniques. Morgan Kaufmann.

[2] Codd, E. F. (1970). A relational model of data for large shared data banks. Communications of the ACM, 13(6), 377–387.

[3] Codd, E. F. (1972). Further normalization of the data base relational model. Data Base Systems, 6, 33–64.

[4] Leber, C., Bilokon, P., & Burgess, M. (2011). Building Low-Latency Applications with C++. Packt Publishing.

[5] Merkle, R. C. (1987). A digital signature based on a conventional encryption function. Advances in Cryptology — CRYPTO ’87, LNCS 293, 369–378.

[6] MySQL 8.0 Reference Manual — InnoDB Locking and Transaction Model. Oracle Corporation. https://dev.mysql.com/doc/refman/8.0/en/innodb-locking.html

[7] React Documentation (18.x). Meta Open Source. https://react.dev

[8] Express.js Documentation (4.x). OpenJS Foundation. https://expressjs.com

[9] PCI Security Standards Council. (2022). PCI DSS v4.0 — Requirements and Testing Procedures.

[10] International Accounting Standards Board. (2003). IAS 2: Inventories. IASB.